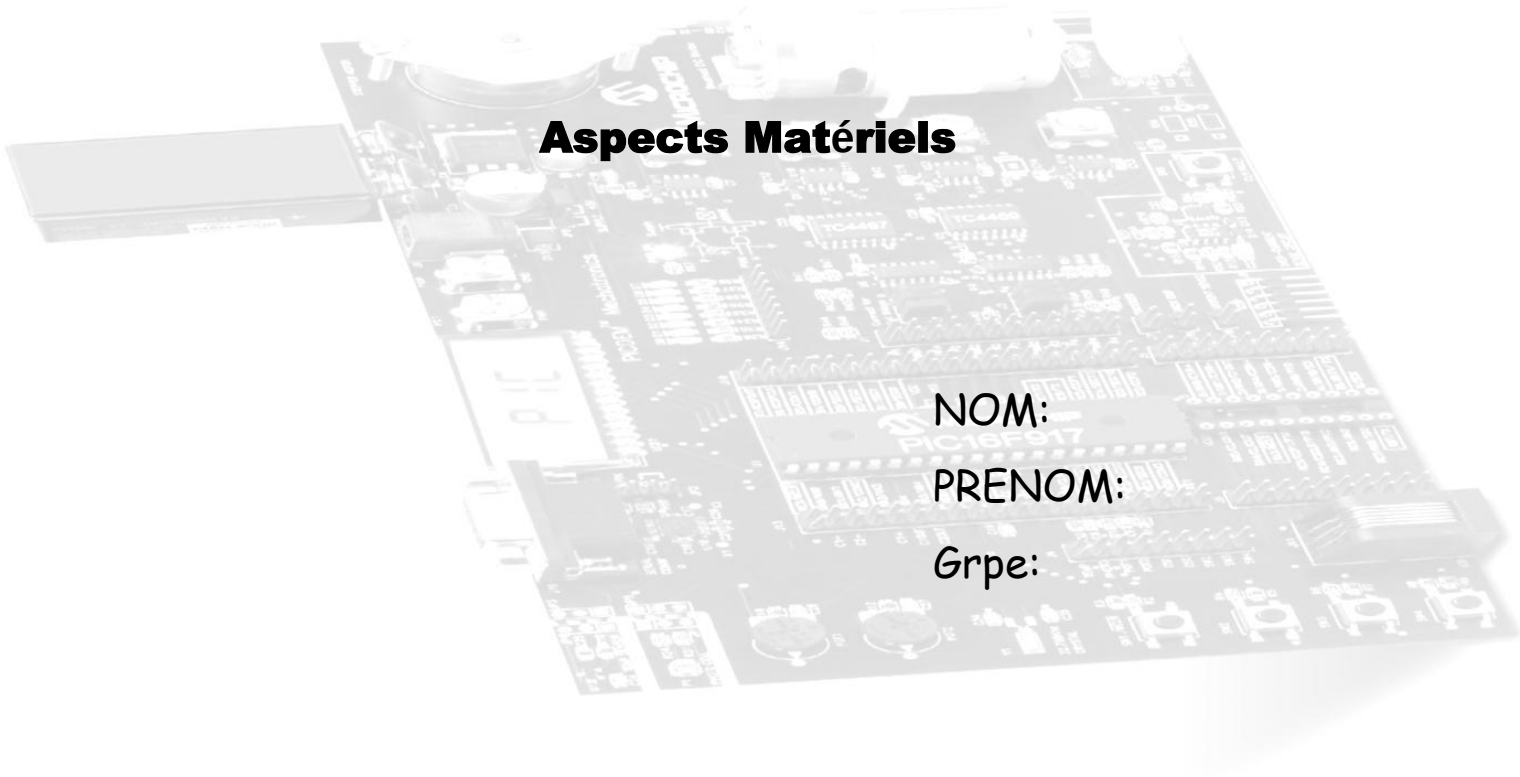
Cours de PIC



Aspects Matériels

NOM:

PRENOM:

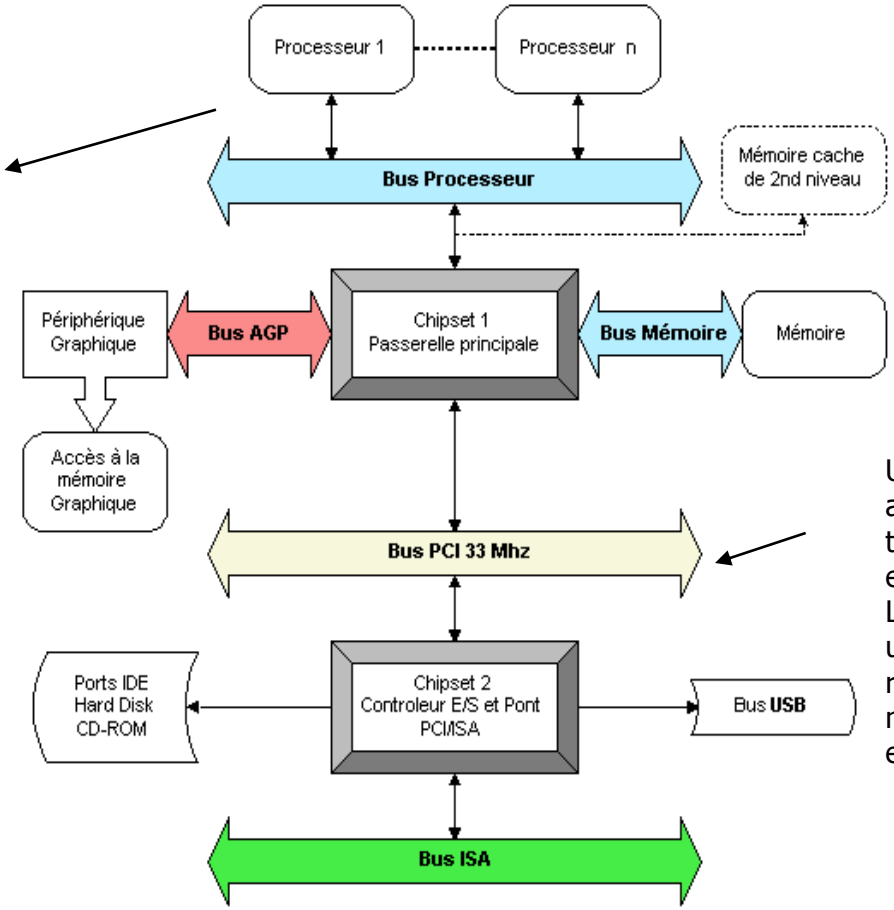
Grpe:

Généralités

Structure des systèmes micro programmé

- Architecture matérielle

Un microprocesseur est un circuit intégré complexe. Il résulte de l'intégration sur une puce de fonctions logiques combinatoires (logiques et/ou arithmétique) et séquentielles (registres, compteur, etc…). Il est capable d'interpréter et d'exécuter les instructions d'un programme

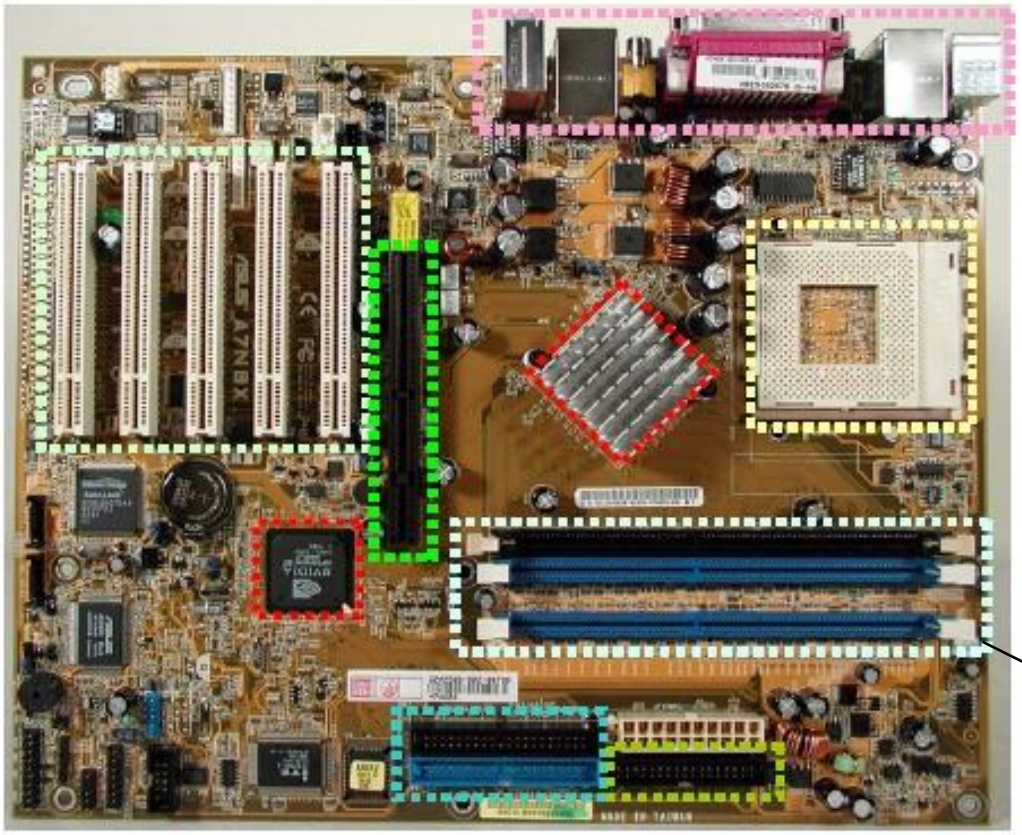


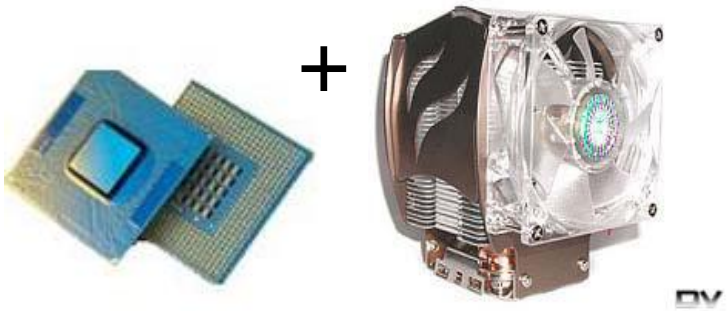
Une mémoire est un circuit à semi-conducteur permettant d'enregistrer, de conserver et de restituer des informations (instructions et variables).

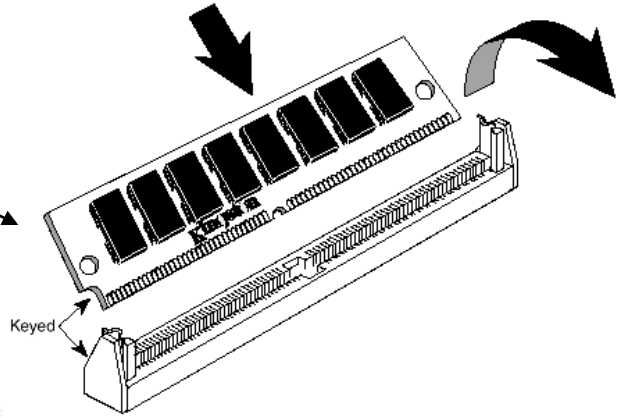
les coupleurs , ou contrôleurs de périphériques, réalisent l'interface électrique et fonctionnelle entre le microprocesseur et les périphériques associés par le biais des différents bus.

Un bus est un ensemble de fils qui assure la transmission du même type d'information. L'information est codée de manière binaire. L'ensemble des 0 et des 1 forment une valeur en fonction du type de représentation adoptée (binaire naturel, représentation signée etc….)

Généralités







Connecteur PCI
Connecteur AGP
Connecteur RAM
Connecteur IDE
Chipset
Socket
Connecteurs Externes (port série, parallèle, firewire, USB, etc…)
Connecteur floppy

Les mémoires

Principe de fonctionnement des mémoires

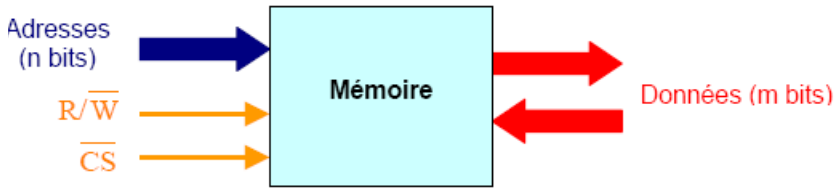
- Objectif : stocker les codes instructions du programme ainsi que les données associées à ce programme

- classification classique suivant 2 familles
  - ✓ RAM : mémoire volatile -> lecture et écriture des données
  - ✓ ROM : mémoire permanente -> lecture uniquement des données (l'écriture correspond à la programmation)

- Principe de fonctionnement
  - ✓ Une mémoire peut être représentée comme une armoire de rangement constituée de différents tiroirs. Chaque tiroir représente alors une case mémoire qui peut contenir un seul élément : des **données.** Le numéro du tiroir que l'on souhaite ouvrir est repéré par un numéro appelé **adresse.**

Adresse	Case mémoire
7 = 111	
6 = 110	
5 = 101	
4 = 100	
3 = 011	
2 = 010	
1 = 001	
0 = 000	0001 1010

bus d'adresses : numéro du tiroir à ouvrir
bus de données : contenu du tiroir
signaux de contrôle : contrôle de l'ouverture de la fermeture



Les mémoires

représentation des nombres

- Représentation en binaire naturel
  - ✓ Les nombres sont exprimés par des chiffres pouvant prendre deux valeurs 0 ou 1. A chaque chiffre est affecté un poids exprimé en puissance de 2

    Ex : $(101)_2 <> 1x\ 2^2 + 0x2^1 + 1x2^0 = (5)_{10}$
- Représentation hexadécimale
  - ✓ Lorsqu'une donnée est représentée sur plus de 4 bits, on préfère souvent l'exprimer en hexadécimal. Les nombres sont exprimés par des chiffres et des lettres pouvant prendre 16 valeurs : 0 1 2 3 4 5 6 7 8 9 A B C D E F .A chaque chiffre est affecté un poids exprimé en puissance de 16.
  - ✓ Ex : $(9A)_{16} <> 9x16^1 + Ax16^0 = 9x16^1 + 10x16^0 = (154)_{10}$
- Représentation signée
  - ✓ Représentations signe et valeur
  - ✓ Représentations en complément un
  - ✓ Représentations en complément à deux

2^6	2^5	2^4	2^3	2^2	2^1	2^0	valeur absolue	complément à 1	complément à 2
1	1	1	1	1	1	1	- 127	- 0	- 1
1	1	1	1	1	1	0	- 126	- 1	- 2
1	1	1	1	1	0	1	- 125	- 2	- 3
1	0	0	0	0	0	1	- 1	- 126	- 127
1	0	0	0	0	0	0	- 0	- 127	- 128
0	1	1	1	1	1	1	+ 127	+ 127	+ 127
0	1	1	1	1	1	0	+ 126	+ 126	+ 126
0	0	0	0	0	0	1	+ 1	+ 1	+ 1
0	0	0	0	0	0	0	+ 0	+ 0	+ 0
Posibilité de représentation							de – 127 à + 127	de – 127 à + 127	de – 128 à + 127

- **En informatique :**
  - ✓ **1 kilo = 210 = 1024**
  - ✓ **1 Mega = 210k = 220= 1024.1024 = 1048576**
  - ✓ **1 Giga = 210M = 220k = 230**
  - ✓ **1 Tera = 210G = 220M = 230k = 240**

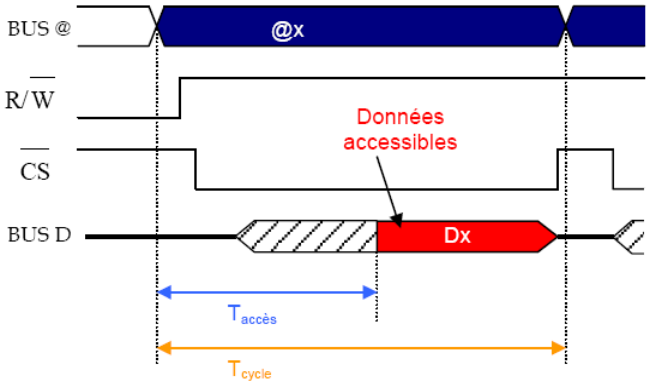
en anglais octet = byte (kb,Mb,...)

Attention : à ne pas confondre avec le bit

Les mémoires

Les memoires ROM

- PROM — programmation par l'utilisateur- contenu figé
- EPROM — *re*programmable effaçable par rayons U.V.
- EEPROM — *re*programmable effaçable électriquement
  - tension de programmation 12,24Volts
- FLASH — *re*programmable effaçable électriquement
  - composant dit ISP
  - Ex : BIOS carte mère PC sur Flash



Exemple de ce cycle de lecture

Les mémoires RAM

- RAM statiques : données mémorisées tant que l'alimentation est présente
  - ✓ mémoire cache externe PC (256/ 512 Ko/1Mo)
  - ✓ Mémoire RAM des micro-contrôleurs.
  - ✓ Registres internes des circuits électroniques (µP,UART,...)
- RAM dynamique : dégradation de l'information au cours du temps =>Rafraîchissement nécessaire
  - ✓ *utilisation intensive dans les PC (EDO, SDRAM, DDRam)*

et le PIC?

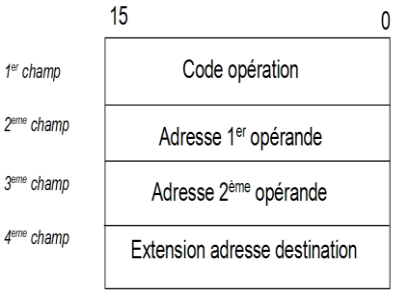
Device	Program Memory		Data Memory	
	Flash (bytes)	# Single-Word Instructions	SRAM (bytes)	EEPROM (bytes)
PIC18F2420	16K	8192	768	256
PIC18F2520	32K	16384	1536	256
PIC18F4420	16K	8192	768	256
PIC18F4520	32K	16384	1536	256

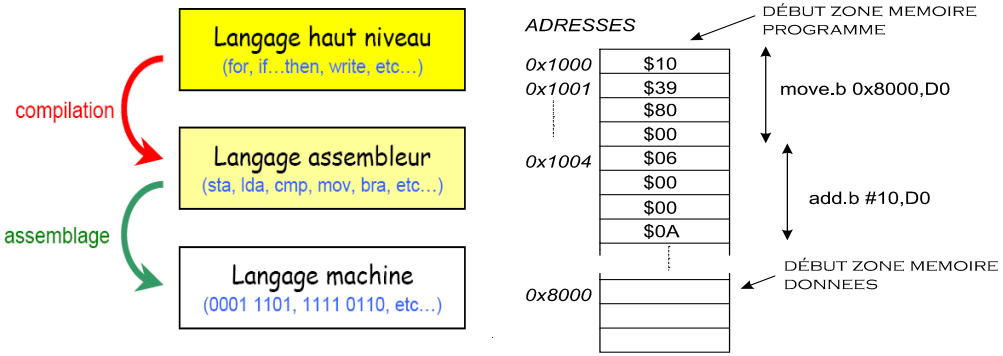
Le microprocesseur et son environnement

Hardware/Software

Software

- ✓ le µP est une sorte d'automate capable d'effectuer à la demande un ensemble limité d'actions
- ✓ chaque action correspond à une instruction du langage compris par le µP
- ✓ Chaque instruction est codée par une série de nombres suivant un format prédéfini
- ✓ Un programme est une suite d'instruction écrite par l'utilisateur
- ✓ Le programme est exécuté instruction par instruction par le µP

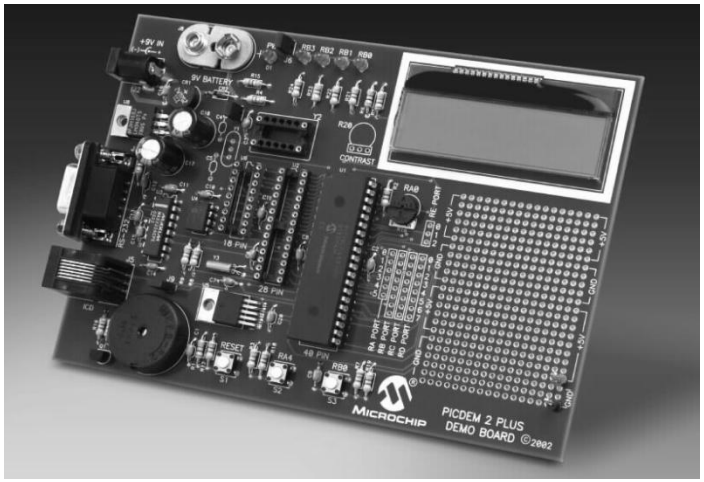




Exemple de codage d'un programme

Hardware

- ✓ partie matérielle du système
- ✓ partie sur laquelle va agir le programme



Le microprocesseur

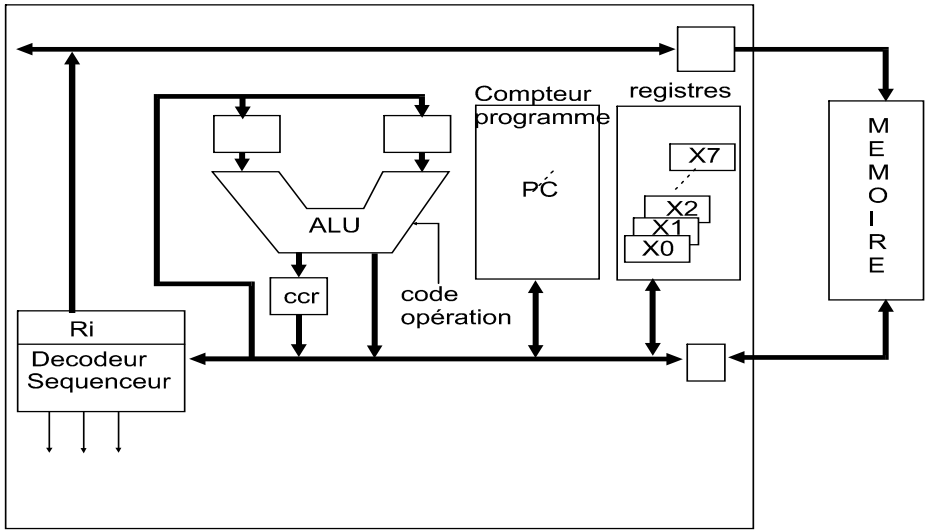
Architecture simplifiée

L'Unité Arithmétique et Logique (UAL) est un circuit complexe qui assure les fonctions logiques (ET, OU, Comparaison, Décalage , etc…) ou arithmétique (Addition, soustraction).

Le registre d'état est généralement composé de 8 bits à considérer individuellement. Chacun de ces bits est un indicateur dont l'état dépend du résultat de la dernière opération effectuée par l'UAL. On les appelle *indicateur d'état* ou *flag* ou *drapeaux*. Dans un programme le résultat du test de leur état conditionne souvent le déroulement de la suite du programme. On peut citer par exemple les indicateurs de :

- o retenue **(carry : C)**
- o retenue intermédiaire
- o signe **(Sign : S)**
- o débordement **(overflow : OV ou V)**
- o zéro **(Z)**
- o parité **(Parity : P)**

le registre d'instruction et le décodeur d'instruction : chacune des instructions à exécuter
est rangée dans le registre instruction puis est décodée par le décodeur d'instruction



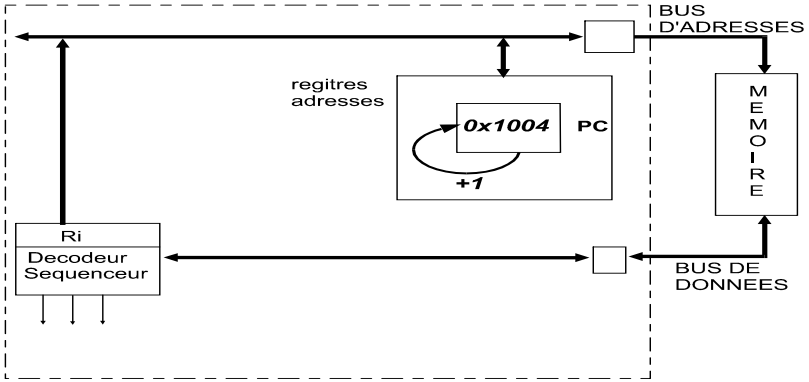
Bloc logique de commande (ou séquenceur) : Il organise l'exécution des instructions au rythme d'une horloge. Il élabore tous les signaux de synchronisation internes ou externes (bus de commande) du microprocesseur en fonction des divers signaux de commande provenant du décodeur d'instruction ou du registre d'état par exemple. Il s'agit d'un automate réalisé soit de façon câblée (obsolète), soit de façon micro-programmée, on parle alors de micromicroprocesseur.

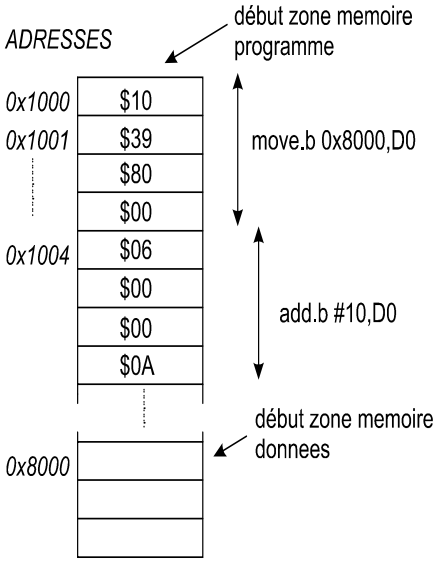
les registres d'usage général permettent à l'unité de traitement de manipuler des données à vitesse élevée. Ils sont connectés au bus données interne au microprocesseur.
les registres d'adresses (pointeurs) connectés sur le bus adresses.

Le microprocesseur

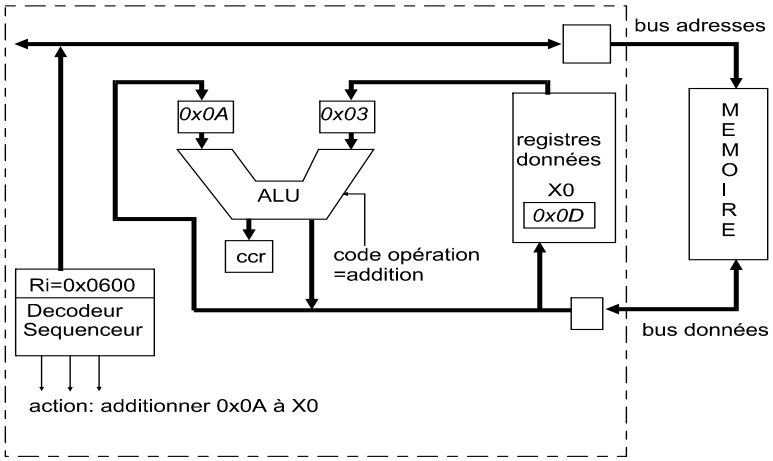
Cycle d'exécution d'une instruction

- Etape 1 => recherche des instructions en mémoire





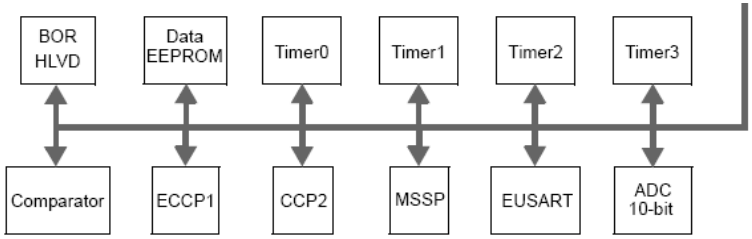
- Etape 2 : décodage de l'instruction
- Etape 3 : Exécution de l'instruction



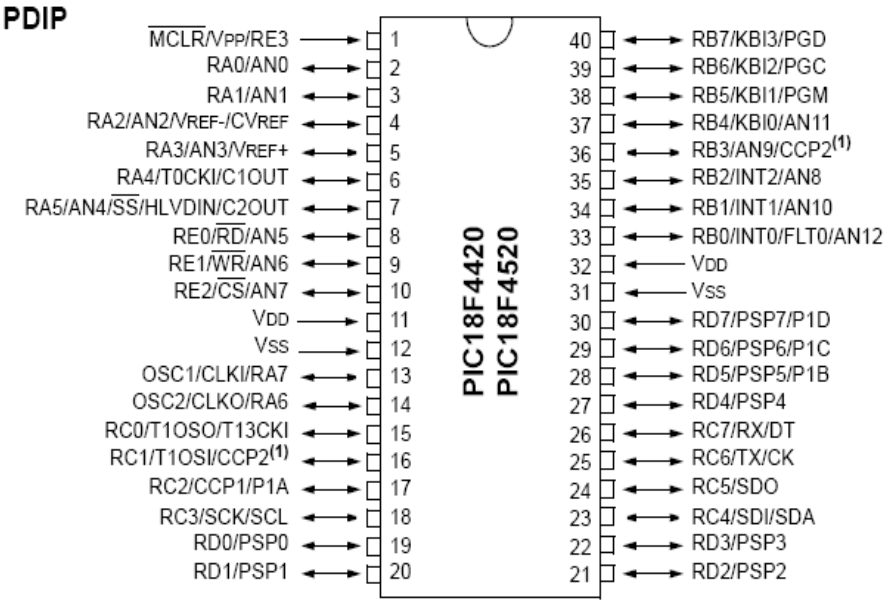
Les PIC 18Fxxxx

Microcontrôleurs

- Un microcontrôleur est un microprocesseur dans lequel on a ajouté au sein du même boîtier
  - ✓ un ensemble de périphériques: convertisseurs analogiques/ numérique et numérique/analogique, timer..
  - ✓ Une quantité limitée de mémoire pour stocker le programme ainsi que des variables



Device	Program Memory		Data Memory	
	Flash (bytes)	# Single-Word Instructions	SRAM (bytes)	EEPROM (bytes)
PIC18F2420	16K	8192	768	256
PIC18F2520	32K	16384	1536	256
PIC18F4420	16K	8192	768	256
PIC18F4520	32K	16384	1536	256



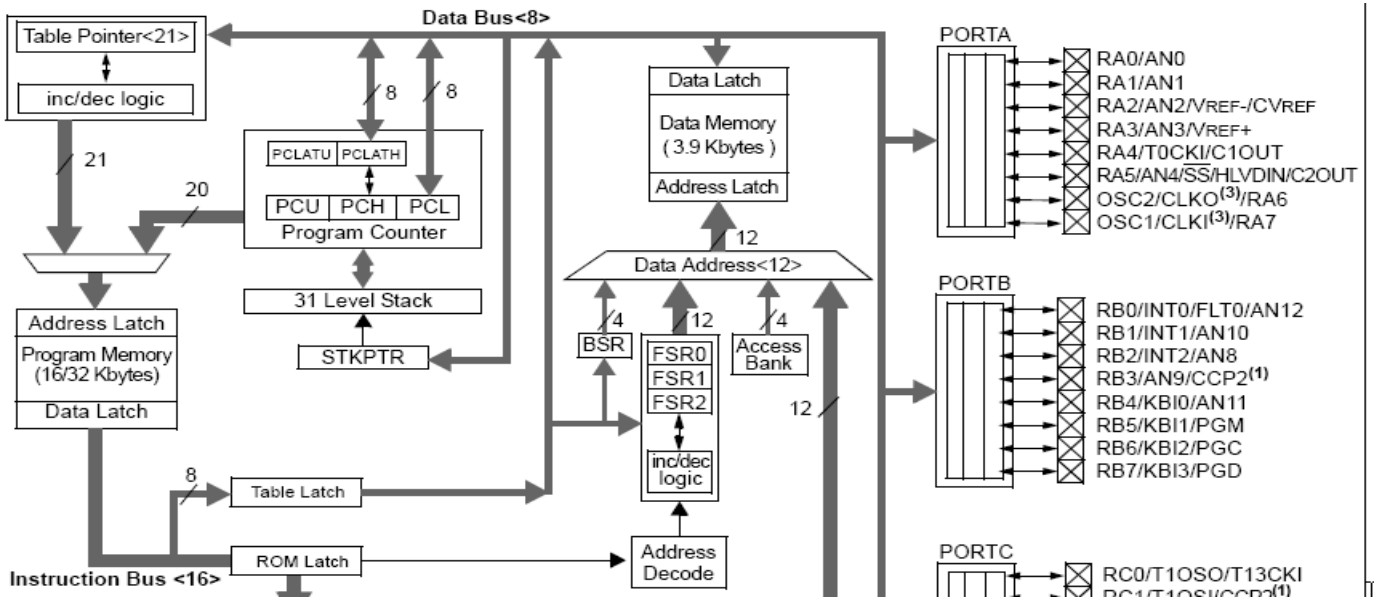
- Développé par Microchip depuis une dizaine d'années, les PIC sont des micro-contrôleurs de seconde génération de type 8 bits externes avec une architecture interne de type **Harvard** autour de son unité centrale permettant des opérations arithmétiques et logiques simples et rapides

Les PIC 18Fxxxx

Architecture du PIC

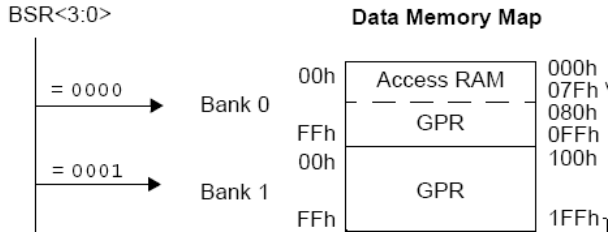
- Les micro-contrôleurs PIC utilisent une **structure dite Harvard** dans laquelle les mémoires Programme et Données sont séparées, adressées toutes deux par deux bus d'adresses différents.Dans ce cas les instructions et les données sont clairement séparées ce qui permet au micro-contrôleur de traiter une donnée en même temps qu'il prépare le cycle de l'instruction suivante

- De plus le micro-contrôleur PIC utilise un **jeu d'instructions réduit** (stocké dans la mémoire programme)
  - ✓ Cette structure est appelée architecture **RISC (Reduced Instruction Set Computer**)

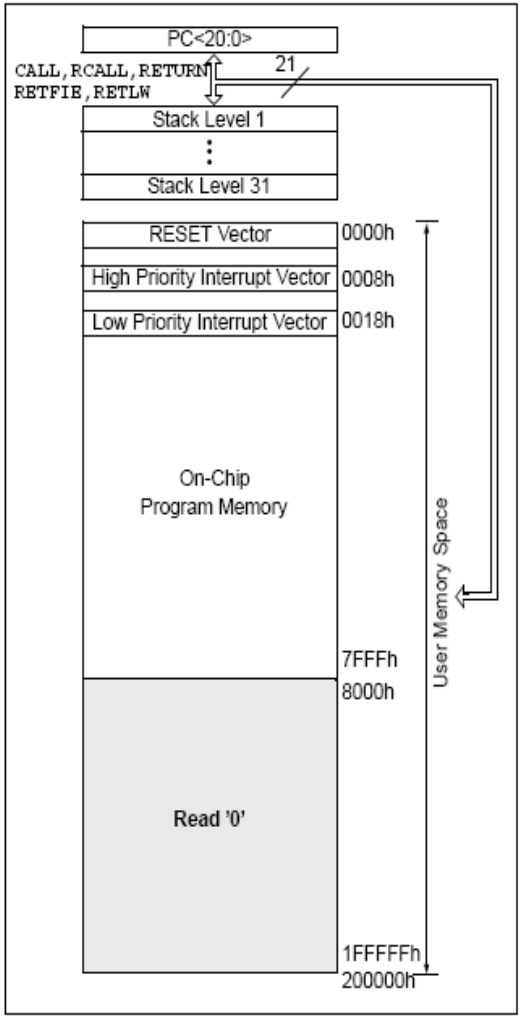


Les PIC 18Fxxxx

- **Mémoire programme interne**
  - Technologie flash
  - 21 bits d'adresses =2 Mo d'espace adressable
  - 32ko implémentés (de 0x0000 à 0x7FFF)
  - Au Reset PC = 0x000000
    - ✓ Première instruction qui est exécutée par le pic au démarrage
  - Les instructions sont codées sur 16 bits (2 cases).
  - Les instructions commencent donc toujours à une adresse paire.
    - ✓ 32ko => 16k instructions
- **Mémoire donnée interne**
  - 12 bits d'adresses 0x000 à 0xFFF et 8 bits de données
  - 16 banques de 256o: bank0 à bank15
    - ✓ bank0 0x000 – 0x0FF
    - ✓ bank1 0x100 – 0x1FF



- La zone 0xF80 – 0xFFF est allouée aux SFR : registre dédiés à une fonction (port E/S, configuration périphériques, ...)

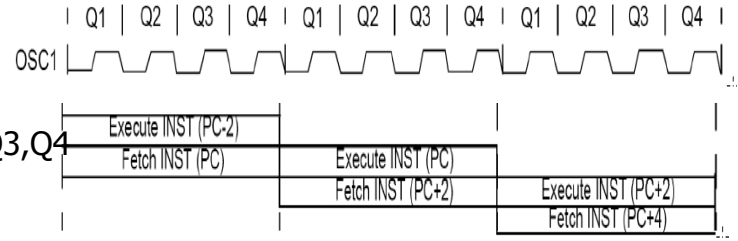


Les PIC 18Fxxxx

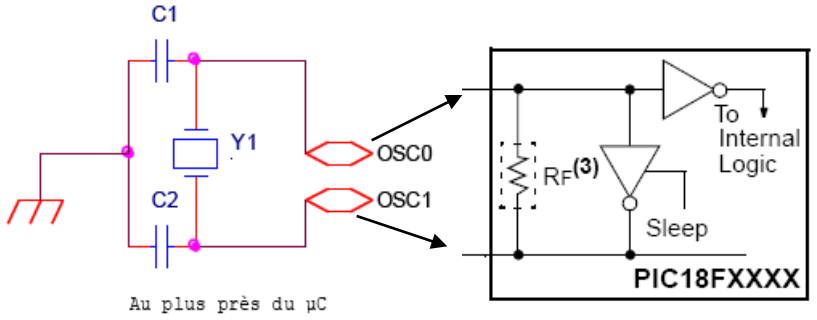
Séquencement des instructions

- architecture de harvard
  - ✓ les instructions et les données sont clairement séparées ce qui permet au micro-contrôleur de traiter une donnée en même temps qu'il prépare le cycle de l'instruction suivante

- Une Horloge qui est un générateur de signaux logiques périodiques stables est nécessaire au fonctionnement des micro-contrôleurs PIC pour cadencer leur fonctionnement instruction après instruction
- la plupart des instructions sont exécutées en un cycle
  - ✓ Le cadenceme
  - ✓ 1cycle dure 4 périodes de l'horloge dénommées : Q1,Q2,Q3,Q4
  - ✓ **Tcy = 4 . Tosc**
  - ✓ Tosc = 1/Fosc avec Fosc : fréquence de l'horloge

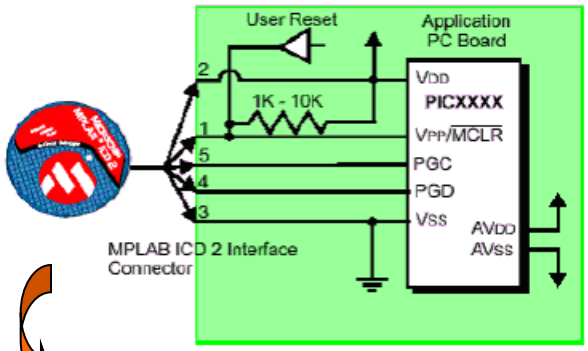


- Différentes possibilités peuvent être utilisées pour réaliser les circuits d'horloge
  - Horloges à quartz soit à basse fréquence (**type LP**) pour économiser en consommation.
  - Horloge à Quartz rapide ou à haute vitesse (**type HS**).
  - Horloge à Quartz ou résonateur standard (**type XT**).
  - Horloge à réseau R-C (**type RC**).
  - Horloge à réseau R-C externe (**type EXTRC**).
  - Horloge à réseau R-C interne (**type INTRC**).

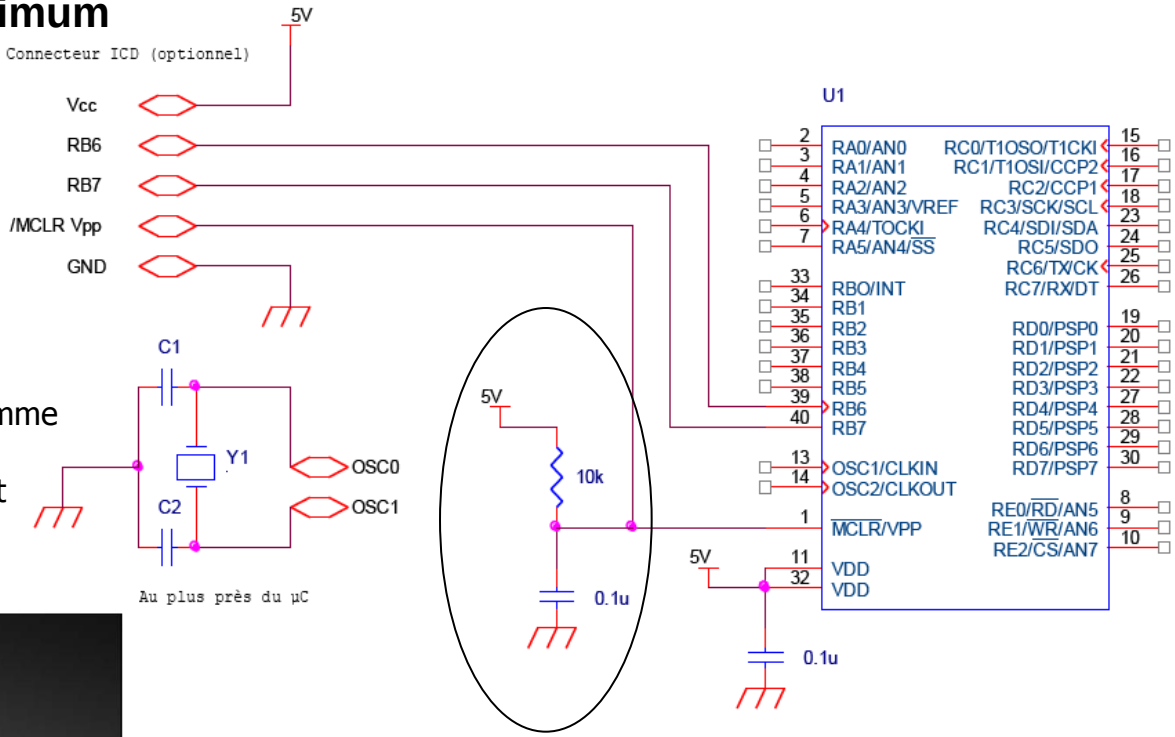


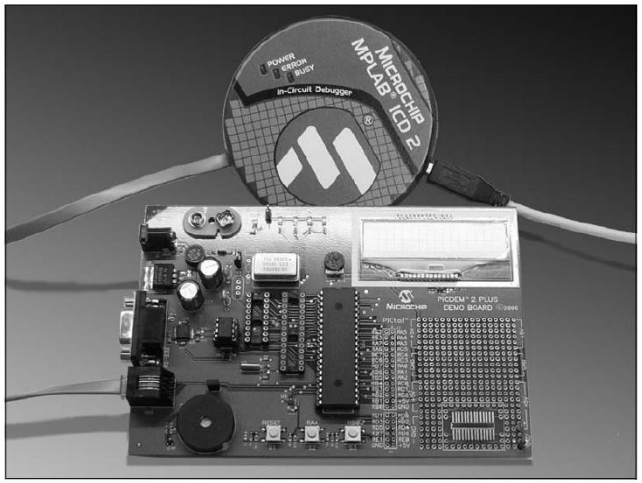
Les PIC 18Fxxxx

schéma de câblage minimum

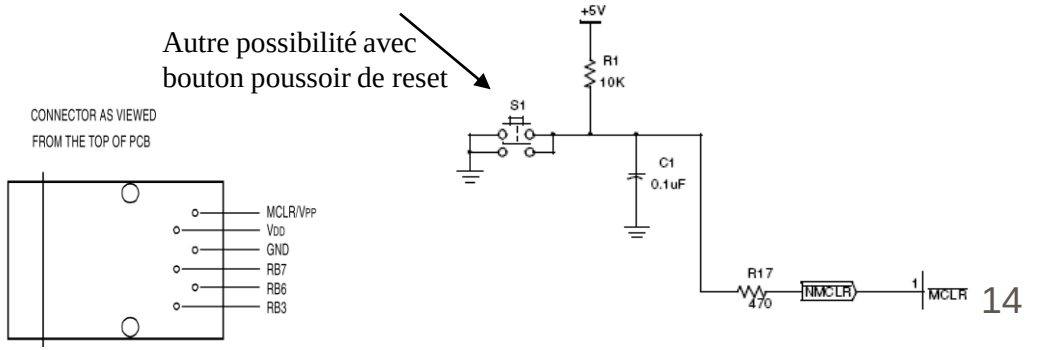


L'ICD2 permet de charger un programme dans le micro (programmeur) et de contrôler son exécution, de voir et de modifier des variables, de placer des points d'arrêt (debugger)





Autre possibilité avec bouton poussoir de reset



Environnement de développement

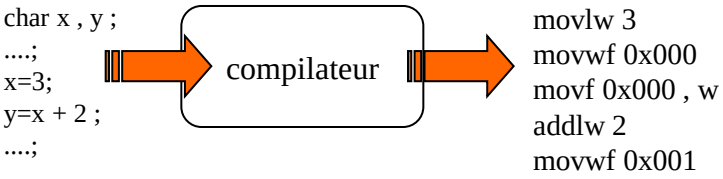
Méthodes de développement

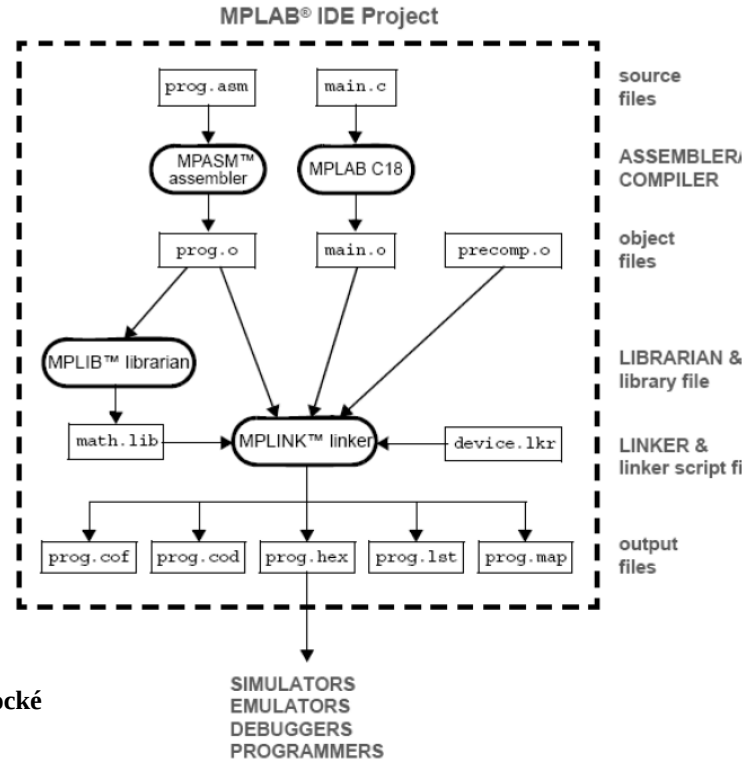
- Développement natif
  - ✓ Le développement logiciel est réalisé sur la carte matérielle ou le programme sera embarqué. Les logiciels de développement sont intégrés dans une mémoire de type rom sur la cible. Ces outils de développement sont en générale rudimentaire.
- Développement croisé (« cross compilation »)
  - ✓ Méthode de développement la plus courante
  - ✓ Le développement logiciel est réalisé sur un PC. Toutes les fonctionnalités d'un PC peuvent être alors utilisées (disques durs accessibles, connexions réseau…)
  - ✓ L'environnement de développement est fourni par le constructeur ou par des entreprises tierces
    - Notre environnement de développement: MPLAB 7.xx fournis par la société MICROCHIP (gratuit!)
      - Comprend un éditeur de texte
      - un compilateur C et assembleur.
      - des outils de mise au point (debugage)
      - les outils de transfert et de programmation du PIC
  - ✓ La plupart des briques logicielles sont développés en C.(seules quelques parties sont écrites en assembleur pour des questions d'optimisation et de rapidité d'exécution du code)

Environnement de développement

La chaîne de développement

- Édition du texte: écriture de votre programme en C.
- Phase de compilation: obtention d'un fichier objet
- Phase d'édition de liens (utilisation du LINKERSCRIPT): permet de lier les différents fichiers objets qui constituent votre projet
- Chargement du programme sur la cible: utilisation de l'ICD2
- Test et mise au point du programme





EXTRAIT DU LINKER SCRIPT 18f4520i.lkr

LIBPATH .

FILES c018i.o **//code de démarrage**

FILES clib.lib **//librairie standard**

FILES p18f4520.lib **//lib librairie spécifique f4520**

CODEPAGE NAME=vectors START=0x0 END=0x29 PROTECTED

CODEPAGE NAME=page START=0x2A END=0x7DBF // adresse où sera stocké votre programme

CODEPAGE NAME=debug START=0x7DC0 END=0x7FFF PROTECTED

CODEPAGE NAME=idlocs START=0x200000 END=0x200007 PROTECTED

CODEPAGE NAME=config START=0x300000 END=0x30000D PROTECTED

…….

Environnement de développement

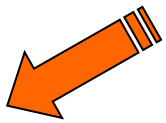
Structure simplifiée d'un programme en C

```
/* titre du programme */                    /*        */     définit du commentaire donc non  compilé.
/* directives de compilation */   //le # précède toutes directives de compilation,
#include "xlcd100.h"  //  inclusion de fichiers : ici  .H nécessaire à l'utilisation de l'afficheur LCD de  la carte PICDEM2+.
#include <p18cxxx.h>  //inclue la librairie p18cxxx.h  qui définie toutes les équivalences entre le nom des registres et leur adresse.
#include <stdio.h>

/* suivant les fonctions système utilisées, inclure les bonnes librairies */
#define S3 PORTAbits.RB3        //PORTAbits.RA4 : représente le bit RA4 du port A, peut être utilisé tel quel, où avec l'étiquette S2
#define   n   0X50              // remplacera toute les étiquettes n par la valeur hexa 0x50

/* configuration du PIC18F 4520 pour l'application sur PICDEM2+ */
# pragma config OSC = HS                    //Oscillateur externe.
# pragma config WDT = OFF                   //Watch Dog inhibé.
# pragma config LVP = OFF                   //Low Voltage ICSP dévalidé.
# pragma config PBADEN =OFF                 //les broches RB0 à  RB4 sont des entrées DIGITALES au reset.
/* déclaration des variables globales (1) */
signed char i, j;                           //variables de type caractère, signées sur 8 bits.
unsigned char c = 9;                        //c sera initialisée à 9 avant l'appel de la fonction main (variable non signée).
signed char tab [20];                       //réservation d'un tableau de 20 caractères.
signed int var1;                            //  variable de type entier, signée sur 16 bits
unsigned char tab1[ ] = "bonjour";          //réservation d'un tableau contenant une  chaîne de caractères
const unsigned char tabcons [ ] = {0xfa,65,'a',22,'r'};  //déclaration d'un tableau de constantes
unsigned char e = 0b11001010                //0b indique que la valeur d'initialisation est donnée en  binaire
/* Ecriture de la fonction main */
void main (void)
{
/* déclaration des variables locales (5) à la fonction main */
unsigned int x;    //entier, variable non signée sur 16 bits.
// mettre le code d'initialisation des périphériques
TRISBbits.TRISB3 = 0 ; // RB3 en sortie
            For( ;;)        {
                            // les taches de votre programme ICI!!!!
                            }

}
```



Le programme embarqué dans le micro a la particularité de démarrer à la mise sous tension du système et de ne s'arrêter qu'à la coupure de l'alimentation. C'est donc un programme qui après une séquence d'initialisation tourne en une boucle infinie.

Environnement de développement

Type d'une variable

- Voir document du compilateur C18

Type	Size	Minimum Exponent	Maximum Exponent	Minimum Normalized	Maximum Normalized
float	32 bits	-126	128	$2^{-126} \approx 1.17549435e-38$	$2^{128} * (2-2^{-15}) \approx 6.80564693e+38$
double	32 bits	-126	128	$2^{-126} \approx 1.17549435e-38$	$2^{128} * (2-2^{-15}) \approx 6.80564693e+38$

Type	Size	Minimum	Maximum
char (1,2)	8 bits	-128	127
signed char	8 bits	-128	127
unsigned char	8 bits	0	255
int	16 bits	-32,768	32,767
unsigned int	16 bits	0	65,535
short	16 bits	-32,768	32,767
unsigned short	16 bits	0	65,535
short long	24 bits	-8,388,608	8,388,607
unsigned short long	24 bits	0	16,777,215
long	32 bits	-2,147,483,648	2,147,483,647
unsigned long	32 bits	0	4,294,967,295

Classes de stockage

- auto : définit une variable de type locale
- extern : déclare une variable sans la crée (la création est dans un autre fichier)
- register : demande au compilateur de faire tout son possible pour utiliser un registre processeur pour cette variable ;
- static : rend une définition de variable persistante
  - ✓ Utilisé dans les interruptions pour mémoriser un état entre deux interruptions
  - ✓ Attention: static ne modifie pas la portée d'une variable locale (la visibilité reste locale mais la durée de vie est 'globale')

Qualificateurs

- Volatile : indique au compilateur une interdiction d'optimisation sur cette variable
- A utiliser lorsque la valeur d'une variable change 'indépendamment' du programme
  - ✓ zone mémoire mappée sur un périphérique: exemple PORT,LAT etc
  - ✓ Variable globale modifiée par une interruption